A Lightweight Academic Research Assistant with LoRA-Tuned Qwen and Semantic Scholar

Tanmay Agarwal
Dept. of Computer Science & Physics
Rider University
Lawrenceville, New Jersey, USA
agarwalt@rider.edu

***Abstract*—This work builds and evaluates a lightweight academic research assistant that converts an informal research topic into an academic search query, retrieves real papers from Semantic Scholar, and reranks the candidates before returning direct links. Qwen2.5-0.5B-Instruct is used only for query rewriting; paper metadata is supplied by Semantic Scholar rather than generated by the model. The model was adapted with Low-Rank Adaptation (LoRA) on 500 custom examples spanning 50 topic families. Only 2.16 million parameters, 0.4359% of the full model, were trainable. Fine-tuning changed the intended behavior: mean rewrite length increased from 4.8 to 10.0 words and the outputs became more search-oriented. Downstream retrieval, however, did not improve overall. Across five fixed queries, mean top-five relevance decreased from 4.68 to 3.72 because some expansions introduced semantic drift, weakened mandatory constraints, or inserted an intent not requested by the user. End-to-end response time changed from 3.967 s to 3.235 s, but the measurement includes external API latency and is therefore not treated as a model-speed result. The main finding is that query rewriting should be optimized for retrieval outcomes and constraint preservation, not for vocabulary richness alone.**

***Index Terms*—information retrieval, query reformulation, LoRA, Qwen2.5, Semantic Scholar, academic search, reranking**

I. Introduction

Academic search often begins with a short, informal description of a topic. A student may know the subject but not the terminology used in the literature, while a search engine still has to distinguish central papers from papers that merely share neighboring concepts. A language model can help reformulate the request, but asking the same model to invent titles, citations, or URLs creates a reliability problem. A plausible citation is not necessarily a real one.

This project separates language understanding from scholarly retrieval. Qwen2.5-0.5B-Instruct rewrites the user's topic into search-oriented terms; Semantic Scholar returns indexed papers and metadata; a local ranker then orders a larger candidate set and returns the best results. The model therefore never has to act as a bibliographic database. The separation also makes failure analysis possible: a weak result can be traced to rewriting, retrieval, or reranking instead of being hidden in one end-to-end generative step.

The study focuses on four questions. **RQ1** asks whether a 0.5B instruction model can be adapted with a small dataset to produce stronger academic search queries. **RQ2** asks whether those rewrites improve the relevance of the top five returned papers. **RQ3** records end-to-end latency before and after fine-tuning. **RQ4** identifies failure modes and the changes required for the next iteration. These questions deliberately separate model behavior from system behavior. A rewrite can look better while retrieval gets worse.

The final prototype runs the query rewriter locally on an Intel Mac, uses an authenticated Semantic Scholar API connection, retrieves twenty candidates for a five-paper request, reranks them with an explainable scoring function, and returns direct links. The most useful result is not simply that LoRA training worked. It is that the first fine-tuning pass exposed a mismatch between what the model was trained to produce and what the search system actually needed.

II. Related Work and Design Rationale

Query expansion and query reformulation are long-standing information-retrieval techniques for reducing vocabulary mismatch. Relevance models, learned reformulation systems, and survey work on query expansion all emphasize the same basic tradeoff: useful added terms can improve recall, while poorly chosen terms can cause topic drift [7]–[9]. This project sits directly in that tradeoff. The target is not a query that sounds more scholarly; the target is a transformation that makes the retriever return better papers.

Parameter-efficient fine-tuning is used because the task is narrow. LoRA freezes the pretrained weights and learns a low-rank update, commonly written as $W' = W + BA$, where A and B are much smaller than the original weight matrix [2]. The deployed base model remains Qwen2.5-0.5B-Instruct [1]; task-specific behavior is stored in a PEFT adapter [3]. This makes it possible to train on a GPU and later run the same adapted model on CPU without storing a second complete model copy.

Semantic Scholar was chosen because it provides an official Academic Graph API with structured metadata and a large literature graph [4]–[6]. The API returns actual indexed records rather than model-generated citations. That design choice keeps the paper corpus current independently of the language model and allows the application to expose verifiable titles, authors, years, abstracts, citation counts, venues, and URLs.

The local ranker is intentionally transparent. Classical lexical ranking systems such as BM25 provide a stronger theoretical foundation for large-scale retrieval [10], while this project

operates on a small candidate pool already returned by Semantic Scholar. The ranker therefore combines direct field overlap with a few metadata signals. Neural semantic reranking is treated as future work rather than hidden inside the current evaluation.

III. Engineering Decisions

Several implementation choices were made to keep the system both defensible as a research prototype and practical on the available hardware. Table I summarizes the main alternatives considered and the consequences of the chosen design. The important pattern is modularity: the language model is used only where it adds value, while retrieval, ranking, and source verification remain explicit components.

The local development environment also shaped the design. The final application runs on an Intel-based MacBook Pro with 16 GB of memory and no NVIDIA CUDA device. That constraint made CPU inference a deployment requirement rather than an optimization after the fact. Training, by contrast, is a one-time operation and was moved to a Tesla T4 in Google Colab. This split between GPU training and CPU inference is common in parameter-efficient workflows and allowed the same adapter to be used in the final local system without changing the architecture.

Reproducibility was handled at the software boundary. The project uses a dedicated Python 3.11 virtual environment, pinned package versions, a project-local Hugging Face cache, a fixed random seed for the dataset split, and a fixed five-query evaluation set. Credentials are kept outside source code in an environment file. These choices do not improve retrieval scores directly, but they make it possible to rerun the same model and pipeline without depending on hidden machine state.

IV. System Architecture and Implementation

Figure 1 is the complete system at a glance. Each box states what enters and what leaves that stage so the reader can understand the full data path without the surrounding text.

The backend is implemented with FastAPI. The main endpoint accepts the raw query, requested result limit, and optional intent. The rewriter loads Qwen lazily and attaches the LoRA adapter with PEFT. Lazy loading is important because the base model has 496,195,456 parameters. At FP32, the raw parameter storage is about 1.98 GB before runtime overhead; repeatedly loading that model for every request would dominate latency. Once initialized, the process reuses the same model. Inference runs with `model.eval()`, `torch.no_grad()`, and deterministic decoding with sampling disabled.

The local deployment uses Python 3.11, PyTorch 2.2.2, Transformers 4.46.3, PEFT 0.13.2, and FastAPI 0.115.5. Model files are stored in a project-local Hugging Face cache. The Semantic Scholar key is read from a local `.env` file rather than hard-coded. HTTP 429 responses trigger bounded retries and respect `Retry-After` when present. Table II summarizes the implementation choices that materially affect reproducibility.

For a requested output size k, the application retrieves

$$N_{cand} = \min(\max(4k, 20), 100). \tag{1}$$

Thus, the evaluated five-paper request starts with twenty Semantic Scholar candidates. Retrieving only five papers and then "reranking" the same five would leave little opportunity to correct weak initial ordering.

The ranker uses the combined original and rewritten query. After lowercasing and stopword removal, Q is the set of remaining query terms, T the title terms, and B the abstract terms. The normalized lexical features are

$$R_t = \frac{|Q \cap T|}{|Q|}, \qquad R_a = \frac{|Q \cap B|}{|Q|}. \tag{2}$$

Semantic Scholar's original result position contributes $S_o = 1/j$ for one-based position j. Citation count C is compressed as $\ln(1 + C)$ so a very highly cited but weakly related paper cannot dominate the score. Recency R_y is bounded over a ten-year window, and A is 1 when an abstract is present. The evaluated score is

$$S(p) = 8R_t + 4R_a + 2S_o + 0.4\ln(1+C) + 0.8R_y + 0.2A. \tag{3}$$

Title and abstract relevance receive the largest weights because earlier testing showed that citation-heavy ranking could promote influential but off-topic papers. A foundational request receives extra citation weight; a research-gap request receives extra recency weight. The formula is heuristic, but every contribution is visible and can be changed independently.

V. Dataset and Fine-Tuning

The training objective is deliberately narrow: *informal research topic → compact academic search terms*. The model is not trained to memorize papers or generate citations. The JSONL dataset contains 500 examples across 50 topic families, with ten variants per topic. Six variants represent normal academic-search phrasing; four encode explicit intent: recent studies, systematic reviews, foundational research, and research gaps. The topics cover AI, cybersecurity, education, psychology, health, economics, environmental science, transportation, robotics, policy, and related areas.

Figure 2 summarizes the complete training pipeline. The figure also makes clear that the five evaluation strings were not copied into the training set, although their subject areas were represented. The test therefore measures new wording within familiar domains rather than unseen-topic generalization.

The final split used 450 training examples and 50 validation examples with seed 42. Training ran on a Tesla T4 in Google Colab, while deployment remained CPU-only. LoRA adapters were attached to `q_proj`, `k_proj`, `v_proj`, and `o_proj`. The main settings are shown in Table III.

Both recorded training and validation losses decreased, which is consistent with the model learning the supervised rewrite pattern without obvious train/validation divergence at those checkpoints. That result is necessary but not sufficient. Loss measures agreement with the target strings; it does not measure whether Semantic Scholar later returns better papers. The retrieval experiment is therefore the more important test.

TABLE I
KEY DESIGN DECISIONS AND THEIR CONSEQUENCES

Decision	Chosen approach	Alternative avoided	Reason and consequence
Scholarly source	Semantic Scholar API	LLM-generated citations or browser scraping	Returns indexed records with structured metadata and stable API access; reduces citation hallucination risk.
Model scale	Qwen2.5-0.5B-Instruct	Multi-billion-parameter local model	Fits CPU deployment on the target Intel Mac while remaining large enough for a narrow rewriting task.
Adaptation method	LoRA / PEFT	Full-parameter fine-tuning	Trains only 0.4359% of parameters, lowering memory and training cost while preserving the base model.
Search strategy	Retrieve 20, then rerank 5	Retrieve exactly 5	Gives the local scoring stage enough candidates to correct weak initial ordering.
Ranking	Transparent weighted score	Opaque neural ranker as the only scorer	Makes failures inspectable and lets relevance dominate citations; semantic reranking can be added later.
Deployment	Local rewriter + external retrieval API	Fully cloud-hosted generative pipeline	Keeps model inference reproducible and avoids recurring generative API cost while still using a current scholarly index.

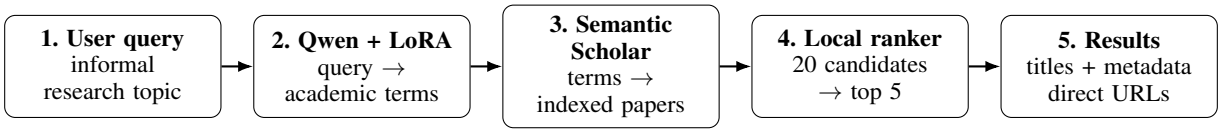


Fig. 1. End-to-end architecture. The language model rewrites the topic but never generates paper records or URLs. Semantic Scholar supplies real records, and the local ranker selects the final five papers.

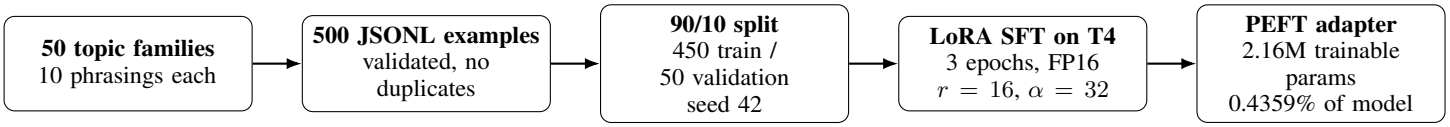


Fig. 2. Training pipeline. The task is query rewriting, not paper memorization. Exact evaluation strings were excluded from training, but related topic families were present.

TABLE II
IMPLEMENTATION SNAPSHOT

Component	Evaluated configuration
Base model	Qwen2.5-0.5B-Instruct
Local inference	Intel CPU, FP32
Training hardware	NVIDIA Tesla T4
Retrieval	Semantic Scholar Academic Graph API
Candidate pool	$N_{cand} = \min(\max(4k, 20), 100)$
Displayed results	Top 5 after reranking
API robustness	Authenticated requests, bounded 429 retry
Model loading	Lazy, one load per server process

TABLE III
FINE-TUNING CONFIGURATION AND RECORDED LOSS

Setting	Value
Epochs	3
Learning rate	2×10^{-4}
Train batch / accumulation	4 / 2
Effective batch	8
Maximum sequence length	256
Precision	FP16
LoRA rank / alpha	16 / 32
LoRA dropout	0.05
Trainable parameters	2,162,688
Trainable fraction	0.4359%
Earlier train / val. loss	0.2349 / 0.2656
Later train / val. loss	0.1022 / 0.1137

The adapter is saved separately from the base model. Deployment loads Qwen and then applies the PEFT adapter, so the effective model can be written conceptually as

$$M_{fine} = M_{base} + \Delta M_{LoRA}. \tag{4}$$

This is one model at inference time, not two independent language models.

VI. Evaluation Method

The before/after comparison uses the same five queries: *AI bias in hiring algorithms*; *sleep academic performance college*; *social media attention span studies*; *climate change policy perspectives*; and *mental health student athletes research gaps*. The exact strings were not training examples, but related topic families were present. This makes the evaluation a controlled test of reformulation behavior within covered domains.

Three measurements are used. First, rewrite behavior is measured by mean output length and exact content-term preservation. Text is lowercased and punctuation removed. Preservation is the proportion of non-stopword terms from the original query that reappear exactly in the rewrite; morphological changes such as *algorithms* $\rightarrow$ *algorithmic* count as non-preservation. Second, each of the top five papers is manually rated from 1 to 5, where 5 means directly on-topic, 4 strongly related with a minor scope difference, 3 partial coverage, 2 a weak connection, and 1 essentially unrelated. The five ratings are averaged for each query. Third, end-to-end response time starts before rewriting and ends after retrieval, local ranking, and response construction. One warm-up request is excluded, then each query is measured three times and averaged.

The relevance assessment is intentionally simple enough to audit from the saved abstracts, but it is not a blinded multi-rater study. The evaluator knew which condition produced each result set. That limitation matters when interpreting small score differences and is addressed in the future-work section.

The evaluation was designed to separate three distinct questions that are easy to conflate. Table IV lists what each metric measures and, equally importantly, what it does *not* establish. This distinction matters because the first training iteration improves the model-level rewrite metrics while reducing the main retrieval metric.

VII. Results

Fine-tuning clearly changed the generated queries. Table V shows all five evaluation rewrites. Mean output length increased from 4.8 to 10.0 words, while exact content-term preservation decreased from 84.3% to 74.3%. The model therefore learned to expand more aggressively, but some original constraints became less stable.

Figure 3 is the central retrieval result. The base system averaged 4.68/5 across the five queries; the fine-tuned system averaged 3.72/5. The sleep query improved from 4.60 to 5.00, climate policy remained at 4.60, and the other three decreased. The largest change was social media/attention span, which fell from 4.80 to 2.20.

The successful sleep case shows what useful expansion looks like. The added terms "sleep quality," "sleep duration," "college students," "cognition," and "memory" remain inside the original information need. All five post-training papers directly examined sleep and academic performance in student populations. The social-media case shows the opposite. "Social comparison" and "psychological wellbeing" are plausible academic neighbors, but they are not equivalent to attention-span research. Once added, Semantic Scholar began returning broader social-media wellbeing literature.

The student-athlete query failed for a different reason. The model generated sensible mental-health terms such as anxiety and depression but weakened two crucial constraints: the population phrase "student athletes" and the intent phrase "research gaps." Several results then concerned students generally rather than student-athletes. The climate query illustrates intent leakage: the rewrite added "recent research" even though the original query did not ask for recency.

Figure 4 reports the measured end-to-end response time. The overall mean was lower in the fine-tuned condition, 3.235 s versus 3.967 s, but the pattern is not consistent across queries and includes external network time. The latency figure is therefore a system observation, not evidence that LoRA itself made local inference faster.

Taken together, the results answer the research questions cleanly. RQ1 is positive: the 0.5B model learned a more consistent academic rewrite style. RQ2 is negative for the first iteration: richer rewrites did not improve average top-five relevance. RQ3 shows mixed system-level latency rather than a causal model-speed effect. RQ4 identifies concrete failure modes that can be acted on rather than a vague statement that "more training" is needed.

Table VI compresses the query-by-query interpretation into one view. It shows that the important variable is not how many terms are added, but whether the expansion preserves the population, outcome, and search intent of the original request.

VIII. Contrasting Retrieval Cases

The aggregate relevance graph is useful, but the difference between a successful and unsuccessful rewrite is even clearer in Figure 5. Both rewrites are longer and more academic than their base versions. The downstream outcome changes only when the added terms change the information need itself.

The sleep rewrite preserves the core population and outcome while adding narrower dimensions of the same topic. "Sleep quality" and "sleep duration" refine the exposure, "college students" preserves the population, and "cognition" and "memory" are mechanisms closely connected to academic performance. The added vocabulary increases the number of ways a relevant paper can match without changing what counts as relevant.

The social-media rewrite behaves differently. "Social comparison" and "psychological wellbeing" are legitimate research topics associated with social-media use, but they compete with "attention span" rather than refine it. A search engine is not aware that one generated term is optional and another is mandatory; all of them can influence matching. The rewrite therefore increases lexical coverage while weakening the relative importance of the user's actual outcome. This explains why the model can improve RQ1 while hurting RQ2.

The same distinction appears in the student-athlete example. "Anxiety" and "depression" are reasonable additions, but the loss of "research gaps" changes the requested task, and weakening "student athletes" changes the population. The practical lesson is

TABLE IV
EVALUATION METRICS AND INTERPRETATION

Metric	Observed comparison	What it measures	What it does not prove
Mean rewrite length	4.8 → 10.0 words	Degree of query expansion	That added terms improve retrieval
Exact term preservation	84.3% → 74.3%	Whether original content terms survive exactly	Semantic equivalence after morphology or synonym replacement
Mean top-5 relevance	4.68 → 3.72	Manual relevance of returned papers to user intent	Universal IR performance from only five queries
End-to-end latency	3.967 → 3.235 s	Complete observed pipeline time	That LoRA itself makes model inference faster
Train/validation loss	0.2349/0.2656 → 0.1022/0.1137	Fit to supervised rewrite targets	Relevance of papers returned by Semantic Scholar

TABLE V
BASE AND FINE-TUNED QUERY REWRITES

Input	Base rewrite	Fine-tuned rewrite
AI bias in hiring algorithms	AI Bias in Hiring Algorithms	algorithmic fairness hiring discrimination employment discrimination machine learning AI
Sleep academic performance college	academic performance sleep college	sleep quality sleep duration academic performance college students cognition memory
Social media attention span studies	social media attention span, study, span	social media social comparison attention span cognitive decline psychological wellbeing
Climate change policy perspectives	climate change policies perspectives	climate change environmental policy mitigation adaptation climate regulation recent research
Mental health student athletes research gaps	mental health, athlete, research, gaps	student mental health athletic performance anxiety depression psychological distress academic research

TABLE VI
QUERY-LEVEL RETRIEVAL OUTCOME AND DIAGNOSED CAUSE

Query	Base	Fine	Observed rewrite behavior	Interpretation
AI hiring bias	5.00	4.00	Added fairness, discrimination, employment, ML	Vocabulary improved, but broadened into general AI fairness; original concise query was already strong.
Sleep / academic performance	4.60	5.00	Added sleep quality, duration, cognition, memory	Added terms stayed inside the original subject and improved retrieval.
Social media / attention	4.80	2.20	Added social comparison, cognitive decline, wellbeing	Neighboring concepts displaced the specific attention-span target; clearest semantic-drift case.
Climate policy	4.60	4.60	Added mitigation, adaptation, regulation, recent research	Mostly useful expansion, but "recent" was unrequested intent leakage; no net relevance change.
Student-athlete mental health gaps	4.40	2.80	Added anxiety/depression but weakened student-athlete and gap constraints	Relevant vocabulary was added while mandatory population and intent constraints were lost.

that query terms are not interchangeable. Some terms describe the topic, some define the population or outcome, and some encode the user's research intent. The next model should preserve those roles instead of flattening all concepts into one bag of keywords.

IX. FAILURE ANALYSIS AND NEXT ITERATION

Figure 6 condenses the three main failure mechanisms. Each mechanism starts with a seemingly reasonable model behavior and ends with a specific retrieval consequence.

The strongest baseline result is also part of the explanation. Semantic Scholar and the local ranker already handled several concise queries very well, producing an overall base relevance of 4.68/5. That creates a ceiling effect. When an original query is already strong, an aggressive rewrite has more room to hurt than to help. The rewriter should therefore be selective rather than mandatory.

The next iteration should treat the original query as authoritative. A useful intermediate representation would separate mandatory concepts from optional expansion terms, for example:

```
required: social media; attention
span
```

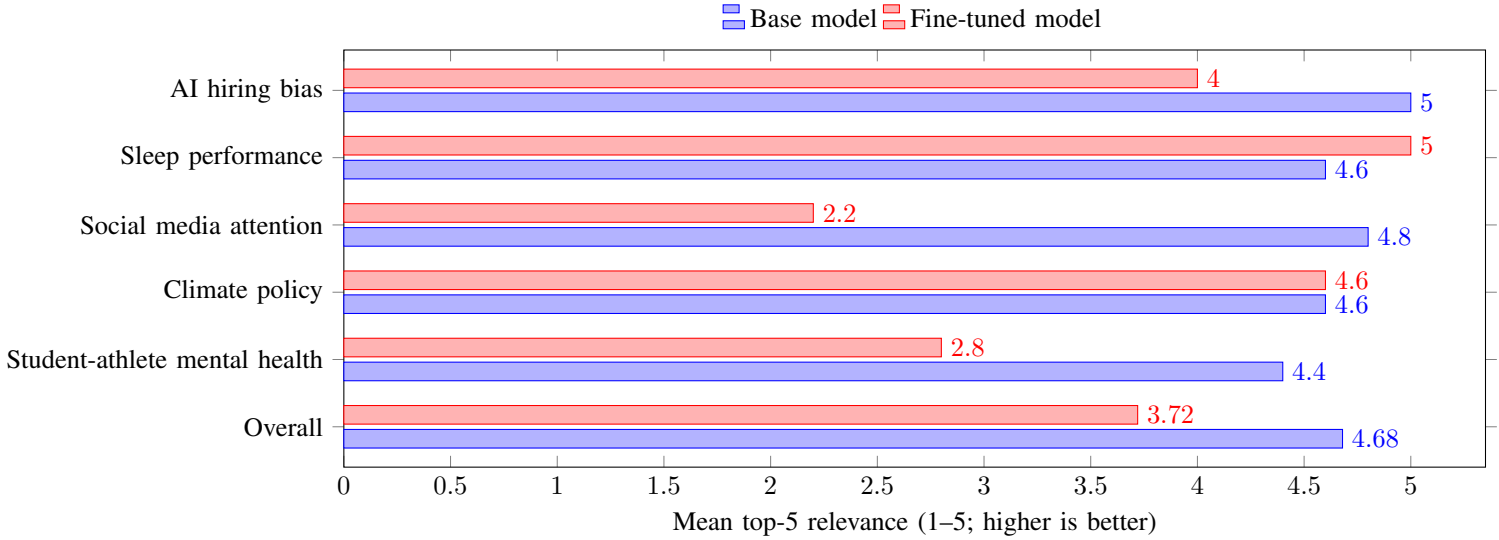


Fig. 3. Top-five paper relevance. Higher bars are better. Fine-tuning improved only the sleep query, matched the base result for climate policy, and reduced the overall mean from 4.68 to 3.72.

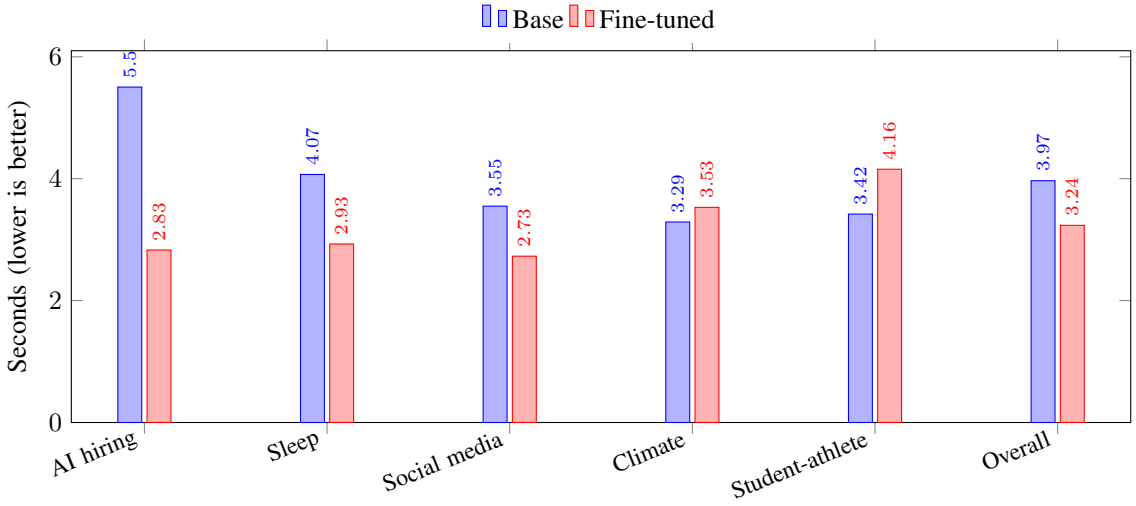


Fig. 4. Measured end-to-end latency. Lower bars are better. Values include local rewriting, Semantic Scholar API/network time, reranking, and response construction; the overall difference is therefore not attributed to LoRA alone.

```
expand: sustained attention;
concentration
intent: general
```

The application can then preserve all required terms while using expansion terms more softly. Training data should include contrastive examples that show when not to add recency, when not to replace a population, and when a well-formed academic query should remain nearly unchanged.

A second improvement is retrieval-aware training. The first dataset rewards linguistically plausible targets. A stronger dataset can compare candidate rewrites by the papers they actually retrieve and prefer the rewrite that improves top-k relevance. Even without reinforcement learning, pairwise preferences between two rewrites of the same query can guide supervised target design.

A practical safeguard is dual search. The system can issue one search with the original query and one with the rewrite, merge and deduplicate the candidate sets, then rerank the union. This reduces the chance that a bad rewrite destroys an already-good original query. It also makes query expansion a recall-enhancing path rather than a single point of failure.

The local ranker can be strengthened with semantic similarity. Sentence-BERT provides an efficient bi-encoder representation [12], while BERT-based cross-encoder reranking can make more precise query-document comparisons [13]. A future hybrid score can retain the explainable current features while adding

$$S'(p) = S(p) + \lambda \,\mathrm{sim}_{embed}(q, p). \tag{5}$$

This would help recover papers that use synonyms and reject

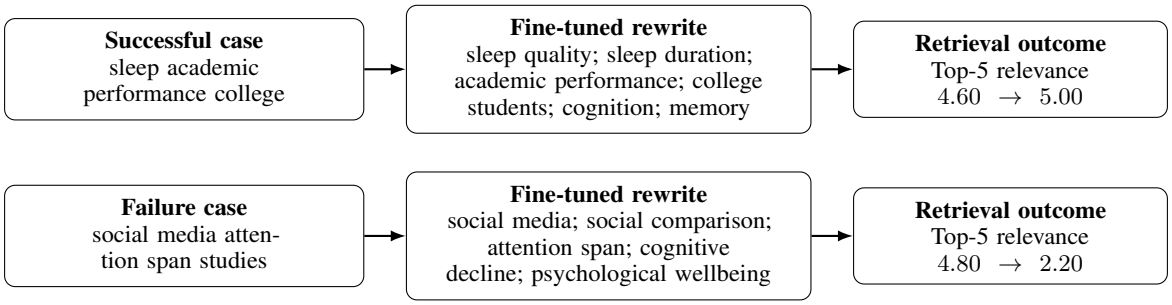


Fig. 5. Two rewrites that both look richer but behave differently in retrieval. The successful case adds dimensions of the same concept; the failure case adds neighboring topics that compete with the original target.

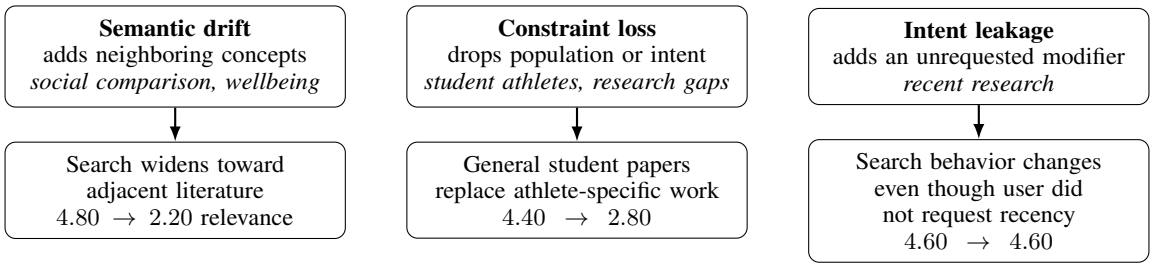


Fig. 6. Observed failure modes and their downstream effect. The problem is not nonsensical vocabulary; it is failure to preserve the user's exact information need while expanding it.

papers that merely match several expansion terms.

The evaluation should also grow. Five fixed queries are enough to expose clear failure mechanisms but not enough for broad claims. Manual relevance scoring was performed by one non-blinded evaluator, so expectancy bias is possible [11]. Semantic Scholar is an evolving index, and end-to-end timing includes network variability. A stronger follow-up should use dozens of queries, blinded or multiple raters, inter-rater agreement, and separate timing for model inference, API calls, and reranking.

X. Validity and Reproducibility

The evaluation is intentionally small and should be read as a controlled prototype experiment rather than a benchmark claim. Five queries are enough to show repeatable differences between the base and fine-tuned rewriters, but they do not represent the full distribution of academic searches. The subject areas also overlap with the training topic families. The exact wording is unseen, but the experiment does not establish generalization to completely new disciplines.

Manual relevance judgments introduce a second limitation. One evaluator assigned all ratings using the same 1–5 rubric and had access to the condition labels. Using one rubric improves consistency, but it does not remove assessor subjectivity; relevance judgments are known to vary between assessors [11]. A stronger evaluation would randomize the result sets, hide whether a set came from the base or fine-tuned model, use multiple raters, and report inter-rater agreement.

The external retriever is another source of variation. Semantic Scholar is an evolving index: new papers can be added, citation counts can change, and ranking behavior can be updated. Exact result lists may therefore differ if the experiment is repeated later. For the same reason, the latency comparison is not a controlled model benchmark. It includes local inference, network transit, Semantic Scholar server time, local reranking, and response construction. Future timing should record these components separately and use many more local inference trials before making any claim about model speed.

Despite these limits, the experiment is reproducible at the application level. The model name, package versions, LoRA configuration, seed, training/validation split, five evaluation queries, candidate-pool rule, ranking equation, and relevance rubric are all explicit. Raw paper metadata and abstracts were preserved during scoring so the relevance judgments can be audited. The resulting numbers should therefore be treated as evidence about this implementation and this test set, not as universal performance estimates.

The system also has a clear reliability boundary. A high relevance score means that a paper appears topically appropriate for the query; it does not establish that the paper is methodologically strong, peer reviewed, or correct. Semantic Scholar reduces one class of hallucination because the returned title and URL correspond to an indexed record, but the index can still contain incomplete metadata, duplicates, preprints, or papers of uneven quality. The application is therefore a discovery tool rather than an automatic literature judge. Users are expected to open the original source and evaluate its evidence before citing it.

Transparency is used as a practical safeguard. The API response keeps both the original query and the rewritten query, which makes it possible to see when the model has inserted a

term such as "recent research" or dropped a constraint such as "research gaps." The ranking function is also inspectable: title overlap, abstract overlap, original result position, citations, recency, and abstract availability all have explicit weights. This matters because the main experimental failure would have been harder to diagnose if both rewriting and reranking were hidden inside a second opaque model.

For a future replication, the experiment can be strengthened without changing the core system. The evaluation queries can be stored in a fixed test file, API outputs can be cached at the time of the run, and each returned paper can be assigned an anonymized identifier before manual scoring. That would separate model-condition knowledge from relevance judgments and reduce the influence of a changing external index. Component-level timing can similarly be recorded as four fields—rewrite, API, rerank, and total—instead of one end-to-end number. These changes would turn the current prototype evaluation into a cleaner repeatable experiment while preserving the same architecture and research question.

XI. Conclusion

This project demonstrates a complete scholarly-search pipeline built around a small local language model rather than a system that asks the model to invent sources. Qwen2.5-0.5B-Instruct was adapted with LoRA on 500 examples while training only 0.4359% of its parameters. The adapter clearly changed the intended behavior: rewrites became longer, more academic, and more consistent. The retrieval experiment then showed why that model-level success was not enough. Mean top-five relevance fell from 4.68 to 3.72 because some expansions drifted away from the user's actual constraints.

The next objective is therefore sharper than simply adding more data or more synonyms. The rewriter should preserve mandatory concepts, avoid unnecessary expansion, and be trained against retrieval outcomes. Dual original/rewrite search and semantic reranking provide practical safeguards around the generative component. The modular architecture makes those changes straightforward: the rewriter, retriever, and ranker can each be improved without rebuilding the complete system. The first iteration is useful precisely because it produces a working prototype and a measurable explanation of what must improve next.

References

[1] Qwen Team, "Qwen2.5 Technical Report," *arXiv preprint arXiv:2412.15115*, 2024.

[2] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," *arXiv preprint arXiv:2106.09685*, 2021.

[3] Hugging Face, "PEFT: Parameter-Efficient Fine-Tuning," Hugging Face Documentation. [Online]. Available: https://huggingface.co/docs/peft/

[4] Semantic Scholar, "Academic Graph API," Semantic Scholar API Documentation. [Online]. Available: https://api.semanticscholar.org/api-docs/

[5] Semantic Scholar, "Semantic Scholar Academic Graph API." [Online]. Available: https://www.semanticscholar.org/product/api

[6] W. Ammar et al., "Construction of the Literature Graph in Semantic Scholar," in *Proc. NAACL-HLT*, 2018, pp. 84–91.

[7] H. K. Azad and A. Deepak, "Query expansion techniques for information retrieval: A survey," *Information Processing & Management*, vol. 56, no. 5, pp. 1698–1735, 2019.

[8] V. Lavrenko and W. B. Croft, "Relevance-Based Language Models," in *Proc. ACM SIGIR*, 2001, pp. 120–127.

[9] R. Nogueira and K. Cho, "Task-Oriented Query Reformulation with Reinforcement Learning," in *Proc. EMNLP*, 2017, pp. 574–583.

[10] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[11] E. M. Voorhees, "Variations in relevance judgments and the measurement of retrieval effectiveness," *Information Processing & Management*, vol. 36, no. 5, pp. 697–716, 2000.

[12] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[13] R. Nogueira and K. Cho, "Passage Re-ranking with BERT," *arXiv preprint arXiv:1901.04085*, 2019.